Spring Security

```java
package com.example.springsecurity;

import org.springframework.security.web.csrf.CsrfToken;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RestController;

import jakarta.servlet.http.HttpServletRequest;

@RestController
public class UserController {


    @GetMapping("/")
    public String sayHello(){
        return "Hello Spring securdfsadfsdfaity";
    }

    @GetMapping("/csrf")
    public CsrfToken getToken(HttpServletRequest request){

        return (CsrfToken) request.getAttribute("_csrf");
    }

    @PostMapping("/add")
    public String addUser(@RequestBody UserModel user){
        System.out.println(user.getName());
        System.out.println(user.getPassword());
        return "User added successfully";
    }

    // jwt authentication
}
```

```java
package com.example.springsecurity;

import lombok.AllArgsConstructor;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

@Getter
@Setter
```

```java
@AllArgsConstructor
@NoArgsConstructor

public class UserModel {

    private String name;
    private String password;

}
```

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>4.1.0</version>
        <relativePath/> <!-- lookup parent from repository -->
    </parent>
    <groupId>com.example</groupId>
    <artifactId>springsecurity</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>springsecurity</name>
    <description/>
    <url/>
    <licenses>
        <license/>
    </licenses>
    <developers>
        <developer/>
    </developers>
    <scm>
        <connection/>
        <developerConnection/>
        <tag/>
        <url/>
    </scm>
    <properties>
        <java.version>21</java.version>
    </properties>
    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-security</artifactId>
```

```
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-webmvc</artifactId>
        </dependency>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-devtools</artifactId>
            <scope>runtime</scope>
            <optional>true</optional>
        </dependency>
        <dependency>
            <groupId>org.projectlombok</groupId>
            <artifactId>lombok</artifactId>
            <optional>true</optional>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-security-test</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-webmvc-test</artifactId>
            <scope>test</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
                <configuration>
                    <excludes>
                        <exclude>
                            <groupId>org.projectlombok</groupId>
                            <artifactId>lombok</artifactId>
                        </exclude>
                    </excludes>
                </configuration>
            </plugin>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-compiler-plugin</artifactId>
                <executions>
                    <execution>
```

```
                    <id>default-compile</id>
                    <phase>compile</phase>
                    <goals>
                        <goal>compile</goal>
                    </goals>
                    <configuration>
                        <annotationProcessorPaths>
                            <path>
                                <groupId>org.projectlombok</groupId>
                                <artifactId>lombok</artifactId>
                            </path>
                        </annotationProcessorPaths>
                    </configuration>
                </execution>
                <execution>
                    <id>default-testCompile</id>
                    <phase>test-compile</phase>
                    <goals>
                        <goal>testCompile</goal>
                    </goals>
                    <configuration>
                        <annotationProcessorPaths>
                            <path>
                                <groupId>org.projectlombok</groupId>
                                <artifactId>lombok</artifactId>
                            </path>
                        </annotationProcessorPaths>
                    </configuration>
                </execution>
            </executions>
        </plugin>
    </plugins>
</build>

</project>
```

📘 Theory

- **Spring Security Basics**
    - Provides authentication, authorization, and protection against attacks.
    - By default, all endpoints are secured with form login.
- **CSRF Protection**
    - Enabled by default in Spring Security.
    - Prevents malicious sites from sending unauthorized requests.

    - CSRF token must be included in POST/PUT/DELETE requests.
- **Authentication & Authorization**
    - Authentication → verifying user identity (username/password).
    - Authorization → checking if the authenticated user has permission to access a resource.
- **UserDetailsService**
    - Interface used to load user-specific data.
    - Custom implementation allows fetching users from DB.
- **Password Encoding**
    - BCryptPasswordEncoder is commonly used to hash passwords securely.
- **Spring JPA + MySQL**
    - Entities mapped with @Entity.
    - JpaRepository provides CRUD operations.
    - Database configured in application.properties.

📝 **Coding Round Questions**

- CSRF token → Write a Spring Boot endpoint that returns the CSRF token to the client.
- Form login → Configure Spring Security to use default form login for authentication.
- Custom user service → Implement a UserDetailsService that loads users from a MySQL database.
- Password encoding → Configure Spring Security to use BCryptPasswordEncoder for password hashing.
- Role-based access → Restrict access to /admin endpoint only for users with role ADMIN.
- Spring JPA CRUD → Implement CRUD endpoints for a Student entity using JPA and MySQL.
- Thymeleaf form → Build a Thymeleaf form that binds to a User object and submits data to a Spring controller.

⚡ **Machine Coding Round Challenge**

Problem: Secure Student Management System (Without JWT)

Requirements:

1. Create a Student entity with fields: id, name, age, course.

2. Implement CRUD endpoints (GET, POST, PUT, DELETE) using Spring JPA and MySQL.
3. Secure endpoints with **Spring Security**:
    - Enable CSRF protection and expose CSRF token via /csrf.
    - Configure form login with default Spring Security login page.
    - Restrict /admin/** endpoints to users with role ADMIN.
4. Add a Thymeleaf form for student registration.
5. Implement global exception handling with @ControllerAdvice.

Expected Flow:

- User logs in via Spring Security form login.
- CSRF token is fetched from /csrf.
- Student data submitted via POST /addStudent with CSRF token.
- Only ADMIN role can delete students.
- Thymeleaf form allows student registration via browser.
- Errors return JSON with timestamp, message, and status.